

Marketing_Efficiency_Suggestion

August 1, 2026

0.1 Marketing Efficiency Suggestion System

*Instead of only predicting marketing efficiency, this notebook **extends the prediction model by recommending the optimal input values that maximize marketing efficiency.***

0.1.1 Import Required Libraries

Import the libraries required for data manipulation, visualization, machine learning, and optimization.

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm
from statsmodels.base.model import GenericLikelihoodModel

from sklearn.linear_model import LinearRegression

from scipy.optimize import minimize
from scipy.stats import norm

from math import sqrt, log, pi
from io import StringIO
```

0.2 Data Preprocessing

Prepare the raw dataset for subsequent analysis by applying sample selection criteria, handling missing values, and performing the initial data cleaning procedures.

0.2.1 Load Dataset

*Load the dataset from a **Parquet** file into a Pandas DataFrame.*

```
[ ]: # Dataset not included in this repository.
# Available upon request for academic collaboration.
```

0.2.2 Create a Working Copy

Create a copy of the original dataset to preserve the raw data during preprocessing and feature engineering.

```
[ ]: merged_df = df.copy()
```

0.2.3 Filter Valid Companies

Keep only companies with at least **two years of observations**, following the sample selection procedure of the original study. This criterion helps avoid **backfilling bias** by including only firms with sufficient historical records before entering the estimation sample (Shi et al., 2022).

Reference

Shi, H., Lim, J., Wang, C., & Zhan, X. (2022). *Marketing Capability, Investor Recognition, and Stock Returns*. *Review of Asset Pricing Studies*, 12(4), 918–960.

```
[ ]: valid_gvkeys = (
    merged_df.groupby('gvkey')['fyear']
    .nunique()
    .loc[lambda x: x >= 2]
    .index
)

merged_df = merged_df[
    merged_df['gvkey'].isin(valid_gvkeys)
]
```

0.2.4 Handle Missing Values

Replace missing values in **advertising expense (XAD)** and **research and development expense (XRD)** with **zero**, assuming that missing values indicate no reported expenditure rather than unavailable observations.

```
[ ]: merged_df['xad'].fillna(0, inplace=True)
merged_df['xrd'].fillna(0, inplace=True)
```

0.2.5 Handle Missing Employee Records

Fill missing **employee (EMP)** values using **forward fill** within each company to preserve the chronological consistency of employee records.

```
[ ]: merged_df['emp'] = (
    merged_df
    .sort_values(['gvkey', 'fyear'])
    .groupby('gvkey')['emp']
    .ffill()
)
```

```
[ ]: merged_df.describe()
```

0.2.6 Filter Invalid Observations

Remove observations with missing or invalid values for *sales (SALE)*, *selling, general and administrative expenses (XSGA)*, and *accounts receivable (RECT)* to ensure reliable estimation and feature construction.

```
[ ]: merged_df = merged_df.dropna(subset=['sale'])
merged_df = merged_df[merged_df['sale'] >= 0]

merged_df = merged_df.dropna(subset=['xsga'])
merged_df = merged_df[merged_df['xsga'] != 0]

merged_df = merged_df.dropna(subset=['rect'])
```

0.2.7 Validate Financial Consistency

Keep only observations where *advertising expenses (XAD)* do not exceed *selling, general, and administrative expenses (XSGA)*, ensuring financially consistent records.

```
[ ]: merged_df = merged_df[
    merged_df['xad'] <= merged_df['xsga']
]
```

0.2.8 Calculate Non-Advertising SG&A Expenses

Separate *non-advertising selling, general, and administrative expenses (XSGA)* by subtracting *advertising expenses (XAD)* from total *SG&A expenses (XSGA)*.

```
[ ]: merged_df['xsga_nonad'] = merged_df['xsga'] - merged_df['xad']
```

0.2.9 Construct Stock Variables

Construct *Advertising Stock (AdStock)*, *SG&A Stock*, and *Sales Stock* using a geometric decay structure to capture the *carryover effect* of past investments, following the methodology adopted in the original study.

```
[ ]: merged_df['adstock'] = (
    merged_df.groupby('gvkey')['xad']
    .transform(lambda x:
        x +
        0.5 * x.shift(1).fillna(0) +
        0.25 * x.shift(2).fillna(0) +
        0.125 * x.shift(3).fillna(0) +
        0.0625 * x.shift(4).fillna(0)
    )
)
```

```
merged_df['sgastock'] = (
    merged_df.groupby('gvkey')['xsga_nonad']
    .transform(lambda x:
        x +
        0.5 * x.shift(1).fillna(0) +
        0.25 * x.shift(2).fillna(0) +
        0.125 * x.shift(3).fillna(0) +
        0.0625 * x.shift(4).fillna(0)
    )
)

merged_df['SalesStock'] = (
    merged_df.groupby('gvkey')['sale']
    .transform(lambda x:
        x +
        0.5 * x.shift(1).fillna(0) +
        0.25 * x.shift(2).fillna(0) +
        0.125 * x.shift(3).fillna(0) +
        0.0625 * x.shift(4).fillna(0)
    )
)
```

0.2.10 Sort Panel Data

Sort the dataset by *company identifier (GVKEY)* and *fiscal year (FYEAR)* to preserve the chronological order required for time-series and panel data calculations.

```
[ ]: merged_df = merged_df.sort_values(['gvkey', 'fyear'])
```

```
[ ]: merged_df.describe()
```

0.2.11 Create Lagged Variables

Create *one-period lagged variables* for stock measures and key financial indicators to ensure that only information from the *previous fiscal year* is used in subsequent modeling, thereby avoiding look-ahead bias.

1 Stochastic Frontier Analysis (SFA)

Prepare the dataset and estimate firms' *marketing efficiency* using a stochastic frontier model based on the methodology of the original study.

```
[ ]: merged_df = merged_df.sort_values(['gvkey', 'fyear'])

merged_df['adstock_lag'] = (
    merged_df.groupby('gvkey')['adstock']
    .shift(1)
)
```

```
merged_df['sgastock_lag'] = (
    merged_df.groupby('gvkey')['sgastock']
    .shift(1)
)

merged_df['SalesStock_lag'] = (
    merged_df.groupby('gvkey')['SalesStock']
    .shift(1)
)

merged_df['rect_lag'] = (
    merged_df.groupby('gvkey')['rect']
    .shift(1)
)

merged_df['xrd_lag'] = (
    merged_df.groupby('gvkey')['xrd']
    .shift(1)
)
```

1.0.1 Create Modeling Dataset

Create a separate copy of the processed dataset to preserve the cleaned data before model development and subsequent analysis.

```
[ ]: model_data = merged_df.copy()
```

1.0.2 Filter Valid Lagged Variables

Retain only observations with **non-negative lagged values** for all stock variables and financial indicators to ensure valid inputs for the stochastic frontier model.

```
[ ]: model_data = model_data[
    (model_data['adstock_lag'] >= 0) &
    (model_data['sgastock_lag'] >= 0) &
    (model_data['SalesStock_lag'] >= 0) &
    (model_data['rect_lag'] >= 0) &
    (model_data['xrd_lag'] >= 0)
].copy()
```

1.0.3 Validate Lagged Variables

Verify that the lagged variables contain **no missing or invalid values** before estimating the stochastic frontier model.

```
[ ]: cols = [
    'adstock_lag',
```

```

    'sgastock_lag',
    'SalesStock_lag',
    'rect_lag'
]

for col in cols:
    print(f"\n{col}")
    print("NaN      :", model_data[col].isna().sum())
    print("<= 0     :", (model_data[col] <= 0).sum())
    print("< 0      :", (model_data[col] < 0).sum())

```

1.0.4 Apply Logarithmic Transformation

Transform the dependent and explanatory variables using the *natural logarithm* to reduce skewness, stabilize variance, and match the logarithmic specification of the stochastic frontier model.

```

[ ]: model_data['ln_adstock_lag'] = np.log(1+ model_data['adstock_lag'])
model_data['ln_sgastock_lag'] = np.log(1 + model_data['sgastock_lag'])
model_data['ln_salesstock_lag'] = np.log(1 + model_data['SalesStock_lag'])
model_data['ln_rect_lag'] = np.log(1 + model_data['rect_lag'])
model_data['ln_xrd_lag'] = np.log(1 + model_data['xrd_lag'])
model_data['ln_sale'] = np.log(1+ model_data['sale'])

```

1.0.5 Validate Logarithmic Variables

Verify that the logarithmically transformed variables contain *no missing or invalid values* before estimating the stochastic frontier model.

```

[ ]: cols = [
    'ln_adstock_lag',
    'ln_sgastock_lag',
    'ln_salesstock_lag',
    'ln_rect_lag' ,
    'ln_xrd_lag',
    'ln_sale'
]

for col in cols:
    print(f"\n{col}")
    print("NaN      :", model_data[col].isna().sum())
    print("<= 0     :", (model_data[col] <= 0).sum())
    print("< 0      :", (model_data[col] < 0).sum())

```

1.0.6 Finalize SFA Dataset

Retain only observations with *valid logarithmic values* to ensure a complete and consistent dataset for stochastic frontier estimation.

```
[ ]: model_data = model_data[
    (model_data['ln_adstock_lag'] >= 0) &
    (model_data['ln_sgastock_lag'] >= 0) &
    (model_data['ln_salesstock_lag'] >= 0) &
    (model_data['ln_rect_lag'] >= 0) &
    (model_data['ln_xrd_lag'] >= 0)&
    (model_data['ln_sale'] >= 0)
].copy()
```

```
[ ]: model_data.describe()
```

1.0.7 Create Industry Classification

Extract the first *two digits* of the SIC code to create **industry groups**, which are used as **industry fixed effects (Industry FE)** in the stochastic frontier model.

```
[ ]: model_data['sic2'] = (
    model_data['sic']
    .astype(str)
    .str[:2]
    .astype(int)
)
```

1.0.8 Create Fixed Effects

Generate **industry** and **year fixed effects** using one-hot encoding to control for unobserved heterogeneity across industries and time periods in the stochastic frontier model.

```
[ ]: industry_fe = pd.get_dummies(
    model_data['sic2'],
    prefix='ind',
    drop_first=True,
    dtype=int
)

year_fe = pd.get_dummies(
    model_data["fyear"],
    drop_first=True,
    dtype=int
)
```

1.0.9 Define Model Variables

Define the dependent variable and construct the design matrix containing the explanatory variables and fixed effects for stochastic frontier estimation.

```
[ ]: X = pd.concat(
    [
```

```

        model_data[
            [
                'ln_adstock_lag',
                'ln_sgastock_lag',
                'ln_salesstock_lag',
                'ln_rect_lag'
            ]
        ],
        industry_fe,
        year_fe ,
        #country_fe ,
        model_data['ln_xrd_lag']
    ],
    axis=1
)

Y = model_data['ln_sale'].values

```

1.1 Estimate the Stochastic Frontier Model

Estimate the stochastic frontier model using *Maximum Likelihood Estimation (MLE)* to obtain the parameter estimates and measure firms' marketing efficiency.

```

[ ]: # =====
# Basic Information
# =====

N = len(Y)
K = X.shape[1]

# =====
# Initial OLS Regression
# =====

reg = LinearRegression()
reg.fit(X.values, Y)

# Initial Parameters
parm = np.concatenate(
    ([reg.intercept_], reg.coef_, [1]),
    axis=0
)

# =====
# Log-Likelihood Function
# =====

```



```

def log_likelihood(params):

    # Separate beta and lambda
    beta = params[:-1]
    lam = params[-1]

    # Predicted values
    y_hat = beta[0] + X @ beta[1:]

    # Residuals
    res = Y - y_hat

    # Sigma squared
    sigma2 = np.sum(res ** 2) / N

    # z
    z = -lam * res / sqrt(sigma2)

    # Standard Normal CDF
    pz = np.maximum(norm.cdf(z), 1e-323)

    # Negative Log Likelihood
    return (
        N / 2 * log(pi / 2)
        + N / 2 * log(sigma2)
        - np.sum(np.log(pz))
        + N / 2
    )

# =====
# Maximum Likelihood Estimation
# =====

result = minimize(
    log_likelihood,
    parm,
    method="L-BFGS-B" ,
    options={
        "maxiter": 10000,
        "maxfun": 100000
    }
)

# =====
# Estimated Parameters
# =====

```

```

params_hat = result.x

beta_hat = params_hat[:-1]

lambda_hat = params_hat[-1]

print("Optimization Success:", result.success)
print(result.message)

print("\nLambda")
print(lambda_hat)

print("\nFirst 10 Betas")
print(beta_hat[:10])

print(result.nit)      # Number of iterations
print(result.nfev)     # Function evaluations
print(result.success)
print(result.message)

```

1.1.1 Retrieve Regression Coefficients

Extract the estimated regression coefficients () from the optimization results for subsequent interpretation and efficiency estimation.

```

[ ]: beta_hat = result.x[:-1]

print(beta_hat)

```

1.1.2 Retrieve the Inefficiency Parameter

Extract the estimated (*lambda*) parameter, which represents the relative contribution of inefficiency to the composite error term in the stochastic frontier model.

```

[ ]: lambda_hat = result.x[-1]

print(lambda_hat)

```

1.1.3 Calculate Model Residuals

Compute the residuals as the difference between the **observed** and **predicted** logarithm of sales. These residuals are subsequently used to estimate firms' marketing efficiency.

```

[ ]: residuals = Y - beta_hat[0] - X @ beta_hat[1:]

print(residuals[:10])

```

1.1.4 Estimate the Residual Variance

Estimate the residual variance (σ^2) from the model residuals. This quantity is subsequently used in the stochastic frontier calculations to estimate inefficiency.

```
[ ]: sigma2 = np.sum(residuals ** 2) / N  
  
print(sigma2)
```

1.1.5 Estimate the Inefficiency Variance

Estimate the variance of the **inefficiency component** (σ_u^2) using the estimated λ parameter and the residual variance. This variance is required to separate inefficiency from random noise in the stochastic frontier model.

```
[ ]: sigma_u2 = (  
    lambda_hat ** 2  
    /  
    (1 + lambda_hat ** 2)  
    ) * sigma2  
  
print(sigma_u2)
```

1.1.6 Estimate the Random Noise Variance

Estimate the variance of the **random noise component** (σ_v^2) using the estimated λ parameter and the residual variance. Together with σ_u^2 , it decomposes the total residual variance into inefficiency and random noise.

```
[ ]: sigma_v2 = sigma2 / (1 + lambda_hat ** 2)  
  
print(sigma_v2)
```

1.1.7 Review Estimated SFA Parameters

Display the estimated λ , σ^2 , σ_u^2 , and σ_v^2 to verify the stochastic frontier parameter estimates before computing firm-level marketing efficiency.

```
[ ]: print(lambda_hat)  
print(sigma2)  
print(sigma_u2)  
print(sigma_v2)
```

1.1.8 Estimate the Conditional Mean of Inefficiency

Estimate the conditional mean of the inefficiency term (u), which represents the expected inefficiency level for each observation based on the estimated residuals and model parameters.*

```
[ ]: ustar = (  
    - residuals
```

```

    * lambda_hat**2
    /
    (1 + lambda_hat**2)
)

```

1.1.9 Estimate the Conditional Standard Deviation

Compute the conditional standard deviation (s) of the inefficiency term, which is required to estimate the expected inefficiency for each observation.*

```

[ ]: sstar = (
    lambda_hat
    /
    (1 + lambda_hat**2)
    * np.sqrt(sigma2)
)

```

1.1.10 Estimate Marketing Efficiency Scores

Estimate the firm-level **marketing efficiency** by applying the conditional expectation of the inefficiency term under the stochastic frontier model. Efficiency scores range between **0 and 1**, where values closer to **1** indicate higher marketing efficiency.

```

[ ]: technical_efficiency = np.exp(
    -ustar
    - sstar
    * (
        norm.pdf(ustar / sstar)
        /
        norm.cdf(ustar / sstar)
    )
)

```

```

[ ]: print(technical_efficiency[:10])

```

1.1.11 Store Marketing Efficiency Scores

Store the estimated **marketing efficiency scores** as a new target variable in the modeling dataset. This variable will be used as the dependent variable in the subsequent machine learning models.

```

[ ]: model_data["marketing_efficiency"] = technical_efficiency

```

1.1.12 Save the Processed Dataset

Save the final modeling dataset, including the estimated **marketing efficiency** scores, for use in the subsequent machine learning and optimization stages.

```

[ ]: model_data.to_parquet("model_data.parquet", index=False)

```

2 Machine Learning

Load the processed dataset containing the estimated **marketing efficiency** scores. This dataset serves as the input for training and evaluating the machine learning models.

```
[ ]: # Dataset not included in this repository.  
# Available upon request for academic collaboration.
```

```
[ ]: model_data.describe()
```

```
[ ]: model_data.isna().sum()
```

2.0.1 Remove Invalid Financial Records

Remove observations with non-positive values for **Total Assets (AT)** and **Common Shares Outstanding (CSHO)** to ensure that all subsequent financial calculations are economically meaningful and mathematically valid.

```
[ ]: model_data = model_data[model_data['at'] > 0]  
model_data = model_data[model_data['csho'] > 0]
```

2.0.2 Select Modeling Variables

Select the final set of financial variables and the target variable (**marketing efficiency**) to construct the machine learning dataset.

```
[ ]: model_data = model_data[[  
    'gvkey',  
    'sale',  
    'xad',  
    'xsga',  
    'xrd',  
    'at',  
    'ni',  
    'rect',  
    'che',  
    'dlc',  
    'dltt',  
    'ppent',  
    'capx',  
    'ceq',  
    'emp',  
    'inv',  
    'cogs',  
    'oancf',  
    'fyear',  
    'prcc_f',  
    'csho',  
    'marketing_efficiency']]
```

```
]]
```

```
[ ]: model_data.head()
```

2.0.3 Prepare the Training Dataset

Create a clean copy of the modeling dataset by removing infinite and missing values to ensure reliable machine learning model training.

```
[ ]: reg_data = model_data.copy()

reg_data.replace([np.inf, -np.inf], np.nan, inplace=True)

reg_data.dropna(inplace=True)
```

2.0.4 Engineer Financial Features

Construct financial ratios and derived variables from the raw accounting data. These engineered features provide a more informative representation of firms' financial characteristics and serve as the primary inputs for the machine learning models.

```
[ ]: reg_data['ln_at'] = np.log(1 + reg_data['at'])

# 4. Profitability (ROA)
reg_data['profitability'] = (
    reg_data['ni'] / reg_data['at']
)

# 5. Leverage
reg_data['leverage'] = (
    (reg_data['dltt'].fillna(0) +
     reg_data['dlc'].fillna(0))
    / reg_data['at']
)

# 6. Cash Holdings
reg_data['cash_holdings'] = (
    reg_data['che'] / reg_data['at']
)

# 7. Tangibility
reg_data['tangibility'] = (
    reg_data['ppent'] / reg_data['at']
)

# 8. Capital Expenditures
reg_data['capex'] = (
    reg_data['capx'] / reg_data['at']
)
```

```

)

# 9. R&D Intensity
reg_data['rd_intensity'] = (
    reg_data['xrd'].fillna(0) / reg_data['sale']
)

# Cash Flow Margin
reg_data['cashflow_margin'] = reg_data['oancf'] / reg_data['sale']

# Inventory Turnover
reg_data['inventory_turnover'] = reg_data['cogs'] / reg_data['invt']

# Equity Ratio
reg_data['equity_ratio'] = reg_data['ceq'] / reg_data['at']

# Market Value
reg_data['market_value'] = reg_data['prcc_f'] * reg_data['csho']

```

2.0.5 Validate Engineered Features

Replace infinite values generated during feature engineering with missing values, then remove incomplete observations to ensure a clean dataset for machine learning model training.

```
[ ]: reg_data.replace([np.inf, -np.inf], np.nan, inplace=True)
reg_data.dropna(inplace=True)
```

```
[ ]: reg_data.columns
```

2.0.6 Define Features, Target, and Groups

Separate the input features, target variable, and grouping variable. The target variable is **marketing efficiency**, while the grouping variable is used to ensure that observations from the same firm remain together during cross-validation.

```
[ ]: X = reg_data.drop(
    columns=[
        "marketing_efficiency",
        "gvkey",
        "fyear",
        "rd_intensity",
    ]
)

y = reg_data["marketing_efficiency"]

groups = reg_data["gvkey"]

```

2.0.7 Import Model Evaluation Tools

Import the cross-validation strategy and evaluation metrics used to train and compare the machine learning models.

```
[ ]: from sklearn.model_selection import GroupKFold
     from sklearn.metrics import (
         r2_score,
         mean_absolute_error,
         mean_squared_error
     )
```

2.0.8 Define Candidate Machine Learning Models

Initialize a diverse set of regression algorithms to compare their predictive performance. The best-performing model will be selected for subsequent hyperparameter tuning and optimization.

```
[ ]: from sklearn.linear_model import LinearRegression, Ridge
     from sklearn.ensemble import RandomForestRegressor
     from xgboost import XGBRegressor
     from lightgbm import LGBMRegressor
     from catboost import CatBoostRegressor

models = {
    "Linear": LinearRegression(),
    "Ridge": Ridge(),
    "RandomForest": RandomForestRegressor(
        random_state=42,
        n_jobs=-1
    ),
    "XGBoost": XGBRegressor(
        random_state=42,
        n_jobs=-1
    ),
    "LightGBM": LGBMRegressor(
        random_state=42
    ),
    "CatBoost": CatBoostRegressor(
        verbose=0,
        random_state=42
    ),
}
```

2.0.9 Compare Candidate Models Using Group Cross-Validation

*Evaluate each regression model using **5-fold Group Cross-Validation**. Model performance is assessed using R^2 , **MAE**, and **RMSE**, and the best-performing model is selected based on its predictive accuracy.*


```

[ ]: gkf = GroupKFold(n_splits=5)

results = []

for name, model in models.items():

    r2_scores = []
    maes = []
    rmses = []

    for train_idx, test_idx in gkf.split(X, y, groups):

        X_train = X.iloc[train_idx]
        X_test = X.iloc[test_idx]

        y_train = y.iloc[train_idx]
        y_test = y.iloc[test_idx]

        model.fit(X_train, y_train)

        pred = model.predict(X_test)

        r2_scores.append(
            r2_score(y_test, pred)
        )

        maes.append(
            mean_absolute_error(y_test, pred)
        )

        rmses.append(
            np.sqrt(
                mean_squared_error(y_test, pred)
            )
        )

    results.append({

        "Model": name,

        "R2": np.mean(r2_scores),

        "MAE": np.mean(maes),

        "RMSE": np.mean(rmses)

    })

```

```

results = pd.DataFrame(results)

results.sort_values(
    "RMSE"
)

```

2.0.10 Hyperparameter Optimization

Optimize the hyperparameters of the **CatBoost Regressor** using **Randomized Search** with **Group K-Fold Cross-Validation**. The objective is to identify the parameter combination that maximizes the model's predictive performance while reducing overfitting.

```

[ ]: from sklearn.model_selection import RandomizedSearchCV, GroupKFold
     from catboost import CatBoostRegressor

param_grid = {
    "depth": [4, 6, 8, 10],
    "learning_rate": [0.01, 0.03, 0.05],
    "iterations": [500, 1000, 1500, 2000],
    "l2_leaf_reg": [3, 5, 7, 9],
}

cv = GroupKFold(n_splits=5)

search = RandomizedSearchCV(
    CatBoostRegressor(verbose=0, random_state=42),
    param_distributions=param_grid,
    n_iter=20,
    scoring="r2",
    cv=cv,
    n_jobs=-1,
    random_state=42
)

search.fit(X, y, groups=groups)

print(search.best_params_)
print(search.best_score_)

```

2.0.11 Train the Final Prediction Model

Train the final **CatBoost Regressor** using the optimal hyperparameters identified during the hyperparameter optimization stage. The model is fitted on the complete dataset to maximize the information available for the final prediction system.

```

[ ]: best_model = CatBoostRegressor(
     learning_rate=0.05,

```

```

    l2_leaf_reg=9,
    iterations=2000,
    depth=8,
    verbose=0,
    random_state=42
)

best_model.fit(X, y)

```

2.0.12 Save the Final Model

Save the trained *CatBoost* model to disk so it can be loaded later for prediction and optimization without retraining.

```
[ ]: best_model.save_model("marketing_efficiency_model.cbm")
```

3 Marketing Efficiency Optimization

Load the trained prediction model to estimate marketing efficiency and generate optimal financial recommendations using optimization techniques.

```
[ ]: # Dataset not included in this repository.
     # Available upon request for academic collaboration.
```

3.0.1 Load the Reference Dataset

Load the final processed dataset containing the engineered financial features. This dataset is used as the reference for defining optimization ranges, computing statistical constraints, and validating the generated recommendations.

```
[ ]: # Dataset not included in this repository.
     # Available upon request for academic collaboration.
```

```
[ ]: best_model.feature_names_
```

3.0.2 Inspect the Reference Dataset

Configure the *DataFrame* display options and inspect a sample observation to verify the structure, feature names, and data values before implementing the optimization workflow.

```
[ ]: import pandas as pd

pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", None)
pd.set_option("display.width", None)
pd.set_option("display.max_colwidth", None)

print(reg_data.iloc[[1]])
```

3.0.3 Define Optimizable Features

Specify the financial variables that the optimization algorithm is allowed to adjust. These features represent managerial decision variables that can potentially be modified to improve the predicted marketing efficiency, while all other variables remain fixed.

```
[ ]: optimizable_features = [  
    "xad",  
    "xsga",  
    "xrd",  
    "rect",  
    "che",  
    "capx",  
    "emp",  
    "inv",  
    "cogs",  
    "oancf"  
]
```

3.0.4 Select a Company for Optimization

Select a single company from the reference dataset as the optimization case. This company's financial information serves as the baseline from which the optimization algorithm searches for improved financial values to maximize the predicted marketing efficiency.

```
[ ]: company = reg_data.drop(  
    columns=[  
        "marketing_efficiency",  
        "gvkey",  
        "fyear"  
    ]  
) .iloc[[45]]
```

3.0.5 Define the Optimization Search Space

Construct the search space for each optimizable feature using summary statistics from the reference dataset. The optimization algorithm searches only within realistic ranges derived from the observed data, ensuring that the generated recommendations remain feasible and economically meaningful.

```
[ ]: # Build the search space for each decision variable  
search_space = {}  
  
for feature in optimizable_features:  
    values = reg_data[feature].dropna()  
    search_space[feature] = {  
        "min": values.quantile(0.05),  
        "max": values.quantile(0.95),  
    }
```

```

    "max": values.quantile(0.95),

    "median": values.median(),

    "std": values.std()

}

```

3.0.6 Estimate Typical Feature Changes

Estimate the typical year-to-year change for each optimizable feature based on historical firm data. These statistics are later used to constrain the optimization process, ensuring that the recommended adjustments remain realistic and consistent with historical business behavior.

```

[ ]: # Estimate historical changes for each decision variable

reg_sorted = reg_data.sort_values(
    ["gvkey", "fyear"]
)

typical_change = {}

for feature in optimizable_features:

    changes = (

        reg_sorted

        .groupby("gvkey")[feature]

        .pct_change()

        .abs()

        .replace([np.inf, -np.inf], np.nan)

        .dropna()

    )

    typical_change[feature] = changes.quantile(0.75)

```

3.0.7 Build Financial Ratio Constraints

Estimate realistic financial ratio ranges from the historical dataset. These ratio-based constraints preserve the financial relationships between variables and prevent the optimization algorithm from generating economically unrealistic recommendations.

```

[ ]: # Financial ratio constraints

ratio_map = {

    "xad": ("sale", "advertising_intensity"),

    "xsga": ("sale", "sga_intensity"),

    "xrd": ("sale", "rd_intensity"),

    "rect": ("sale", "receivable_ratio"),

    "che": ("at", "cash_holdings"),

    "capx": ("at", "capex_ratio"),

    "cogs": ("sale", "cogs_margin"),

    "oancf": ("sale", "cashflow_margin"),

    "inv": ("sale", "inventory_ratio")

}

ratio_limits = {}

for feature, (denominator, ratio_name) in ratio_map.items():

    ratio = (
        reg_data[feature] /
        reg_data[denominator]
    )

    ratio = (
        ratio
        .replace([np.inf, -np.inf], np.nan)
        .dropna()
    )

    ratio_limits[feature] = {

        "p5": ratio.quantile(0.05),

        "p10": ratio.quantile(0.10),

        "median": ratio.quantile(0.50),
    }

```

```

    "p90": ratio.quantile(0.90),

    "p95": ratio.quantile(0.95)

}

```

3.0.8 Define the Optimization Objective Function

Define the objective function used by **Optuna** to maximize the predicted marketing efficiency. For each optimization trial, candidate financial values are generated within realistic search ranges, derived financial features are recalculated, business constraints are enforced, and the trained Cat-Boost model evaluates the resulting marketing efficiency. The predicted efficiency serves as the optimization objective.

```

[ ]: import optuna
import numpy as np

def safe_div(a, b):
    return a / b if b != 0 else 0.0

def objective(trial):

    temp = company.copy()

    # =====
    # Generate Candidate Values
    # =====

    for feature in optimizable_features:

        current = float(company.iloc[0][feature])

        change = typical_change[feature]

        if current <= 1e-8:

            lower = max(0.0, search_space[feature]["min"])
            upper = max(lower + 1e-6, search_space[feature]["median"])

        else:

            lower = current * (1 - change)
            upper = current * (1 + change)

            lower = max(lower, search_space[feature]["min"])
            upper = min(upper, search_space[feature]["max"])

```

```

        lower = max(lower, 0.0)

    if lower >= upper:

        lower = max(0.0, search_space[feature]["min"])
        upper = max(lower + 1e-6, search_space[feature]["max"])

    temp.loc[temp.index[0], feature] = trial.suggest_float(
        feature,
        lower,
        upper
    )

# =====
# Read Variables
# =====

at = temp.iloc[0]["at"]
sale = temp.iloc[0]["sale"]
ni = temp.iloc[0]["ni"]

dlc = temp.iloc[0]["dlc"]
dltt = temp.iloc[0]["dltt"]

che = temp.iloc[0]["che"]
ppent = temp.iloc[0]["ppent"]
capx = temp.iloc[0]["capx"]

cogs = temp.iloc[0]["cogs"]
invnt = temp.iloc[0]["invnt"]

oancf = temp.iloc[0]["oancf"]

ceq = temp.iloc[0]["ceq"]

prcc_f = temp.iloc[0]["prcc_f"]
csho = temp.iloc[0]["csho"]

xad = temp.iloc[0]["xad"]
xsga = temp.iloc[0]["xsga"]
xrd = temp.iloc[0]["xrd"]
rect = temp.iloc[0]["rect"]

# =====
# Recalculate Derived Variables
# =====

```



```

temp.loc[temp.index[0], "ln_at"] = np.log(max(at, 1e-8))

temp.loc[temp.index[0], "profitability"] = safe_div(ni, at)

temp.loc[temp.index[0], "leverage"] = safe_div(dltt + dlc, at)

temp.loc[temp.index[0], "cash_holdings"] = safe_div(che, at)

temp.loc[temp.index[0], "tangibility"] = safe_div(ppent, at)

temp.loc[temp.index[0], "capex"] = safe_div(capx, at)

temp.loc[temp.index[0], "cashflow_margin"] = safe_div(oancf, sale)

temp.loc[temp.index[0], "inventory_turnover"] = safe_div(cogs, invt)

temp.loc[temp.index[0], "equity_ratio"] = safe_div(ceq, at)

temp.loc[temp.index[0], "market_value"] = prcc_f * csho

# =====
# Business Ratios
# =====

business_ratios = {

    "xad": safe_div(xad, sale),

    "xsga": safe_div(xsga, sale),

    "xrd": safe_div(xrd, sale),

    "rect": safe_div(rect, sale),

    "che": safe_div(che, at),

    "capx": safe_div(capx, at),

    "cogs": safe_div(cogs, sale),

    "oancf": safe_div(oancf, sale),

    "invt": safe_div(invt, sale)

}

```

```

# =====
# Business Constraints
# =====

#penalty = 0

for feature, ratio in business_ratios.items():

    limits = ratio_limits[feature]

    if ratio < limits["p5"]:
        return -1e9

    if ratio > limits["p95"]:
        return -1e9
# =====
# Predict Marketing Efficiency
# =====

prediction = best_model.predict(temp)[0]

trial.set_user_attr("prediction", float(prediction))

return prediction

```

3.0.9 Run the Optimization Process

Execute the optimization procedure using *Optuna's Tree-structured Parzen Estimator (TPE)* to search for the financial values that maximize the predicted marketing efficiency. The optimized prediction is then compared with the current prediction to quantify the expected improvement and identify the recommended financial adjustments.

```

[ ]: import optuna

sampler = optuna.samplers.TPESampler(seed=42)

study = optuna.create_study(
    direction="maximize",
    sampler=sampler
)

study.optimize(
    objective,
    n_trials=500
)

```

```

best_prediction = study.best_value
best_params = study.best_params

current_prediction = best_model.predict(company)[0]

if best_prediction <= current_prediction:
    optimized_prediction = current_prediction
    improvement_percent = 0
    recommended_values = {}
else:
    optimized_prediction = best_prediction
    improvement_percent = (
        (optimized_prediction - current_prediction)
        / current_prediction
    ) * 100
    recommended_values = best_params

```

3.0.10 Predict the Current Marketing Efficiency

Estimate the company's current marketing efficiency using the trained CatBoost model. This prediction serves as the baseline for evaluating the effectiveness of the optimization process.

```
[ ]: current_efficiency = best_model.predict(company)[0]

print(current_efficiency)
```

3.0.11 Retrieve the Reference Marketing Efficiency

Retrieve the original marketing efficiency score estimated by the Stochastic Frontier Analysis (SFA). This value serves as the reference benchmark for comparing the machine learning prediction and the optimized recommendation.

```
[ ]: reg_data.iloc[[45]]["marketing_efficiency"]
```

3.0.12 Retrieve the Optimal Financial Recommendations

Retrieve the optimal values identified by the optimization algorithm for all decision variables. These values represent the recommended financial adjustments that maximize the predicted marketing efficiency while satisfying all optimization constraints.

```
[ ]: study.best_params
```

3.0.13 Apply the Optimized Financial Recommendations

Create a copy of the selected company and update its decision variables using the optimal values identified by the optimization process. The resulting dataset represents the recommended financial scenario that will be evaluated by the prediction model.

```
[ ]: recommended_company = company.copy()

for feature, value in study.best_params.items():

    recommended_company.loc[
        recommended_company.index[0],
        feature
    ] = value
```

```
[ ]: recommended_efficiency = best_model.predict(
    recommended_company
)[0]

print(recommended_efficiency)
```

```
[ ]: recommended_company
```

3.0.14 Compare the Current and Recommended Financial Values

Generate a comparison table showing the current financial values, the optimized recommendations, and the percentage change for each decision variable. This summary provides a clear interpretation of the optimization results and highlights the suggested financial adjustments.

```
[ ]: comparison = []

for feature in optimizable_features:

    current = company.iloc[0][feature]

    recommended = recommended_company.iloc[0][feature]

    comparison.append({

        "Feature": feature,

        "Current": current,

        "Recommended": recommended,

        "Change %": (
            (recommended-current)
            /
            max(abs(current), 1e-6)
        ) * 100

    })

comparison = pd.DataFrame(comparison)
```

```
comparison
```

3.0.15 Evaluate the Optimization Results

Compare the predicted marketing efficiency before and after applying the optimized financial recommendations. The percentage improvement quantifies the expected performance gain achieved through the proposed financial adjustments.

```
[ ]: current_prediction = best_model.predict(company)[0]

optimized_prediction = best_model.predict(recommended_company)[0]

improvement = (
    (optimized_prediction - current_prediction)
    / current_prediction
) * 100

print(current_prediction)
print(optimized_prediction)
print(improvement)
```

3.0.16 Save the Optimization Configuration

Save the optimization configuration, including the search space, typical feature changes, and business ratio constraints. These files enable the recommendation system to reproduce the same optimization process without recalculating the supporting statistics.

```
[ ]: import joblib

joblib.dump(
    search_space,
    "search_space.pkl"
)

joblib.dump(
    typical_change,
    "typical_change.pkl"
)

joblib.dump(
    ratio_limits,
    "ratio_limits.pkl"
)
```

3.0.17 Save the Feature Configuration

Save the ordered list of input features used during model training. Preserving the feature order ensures that future prediction and optimization processes provide the trained model with inputs that match its expected structure.

```
[ ]: import joblib

feature_columns = [
    'sale',
    'xad',
    'xsga',
    'xrd',
    'at',
    'ni',
    'rect',
    'che',
    'dlc',
    'dltt',
    'ppent',
    'capx',
    'ceq',
    'emp',
    'inv',
    'cogs',
    'oancf',
    'prcc_f',
    'csho',
    'ln_at',
    'profitability',
    'leverage',
    'cash_holdings',
    'tangibility',
    'capex',
    'cashflow_margin',
    'inventory_turnover',
    'equity_ratio',
    'market_value'
]

joblib.dump(
    feature_columns,
    "feature_columns.pkl"
)
```

4 Conclusion

*This work presents an integrated decision support framework that combines **Stochastic Frontier Analysis (SFA)**, **Machine Learning (CatBoost)**, and **Bayesian Optimization (Optuna)** to move beyond prediction toward actionable financial recommendations. The proposed system estimates marketing efficiency, identifies optimal financial adjustments under realistic business constraints, and provides interpretable recommendations that can support managerial decision-making.*

5 Framework

The proposed framework extends traditional marketing efficiency prediction by integrating econometric analysis, machine learning, and optimization into a unified decision support system. Rather than only estimating marketing efficiency, the framework recommends optimal financial adjustments that maximize the predicted marketing efficiency while satisfying realistic financial and business constraints.

Name : Karim Maher

Email: karim.henish@gmail.com

Phone: +20 1067220050

LinkedIn: <https://www.linkedin.com/in/kari-m-aher/>

Live Demo: <https://marketing-efficiency-lab.web.app/>

GitHub Repository: <https://github.com/karimhenish/marketing-efficiency-lab>

[]: